

Multi-tenant SaaS Platform Testing Documentation

Table of Contents

1. Introduction
2. Testing Environment Setup
3. Installation Testing
4. Configuration Testing
5. API Endpoint Testing
6. Frontend Testing
7. Multi-tenancy Testing
8. Database Testing
9. Security Testing
10. Performance Testing
11. Integration Testing
12. Regression Testing
13. Test Case Templates
14. Automated Testing

Introduction

This document provides comprehensive testing procedures for the Multi-tenant SaaS Platform. It covers all aspects of testing, from installation verification to security and performance testing. The procedures are designed to ensure that the system functions correctly, securely, and efficiently in all scenarios.

Testing is a critical part of maintaining the quality and reliability of the SaaS Platform. This document serves as a guide for quality assurance engineers, developers, and system administrators who need to verify the system's functionality.

Testing Objectives

The main objectives of testing the SaaS Platform are:

1. Verify that all components install and configure correctly
2. Ensure that all API endpoints function as expected
3. Validate that the frontend UI works properly
4. Confirm that multi-tenancy features provide proper data isolation
5. Test database operations and data integrity
6. Verify security measures and authentication mechanisms
7. Assess system performance under various load conditions
8. Ensure compatibility across different environments

Testing Methodology

The testing approach follows these principles:

1. **Systematic Coverage:** Test all components and features systematically
2. **Incremental Testing:** Start with unit tests, then integration tests, and finally system tests
3. **Realistic Scenarios:** Use real-world scenarios to validate functionality
4. **Automated Where Possible:** Automate repetitive tests for efficiency
5. **Detailed Documentation:** Document all test cases and results

Testing Environment Setup

Before beginning testing, you need to set up a proper testing environment that is isolated from production systems.

Hardware Requirements

- **CPU:** 2+ cores recommended
- **RAM:** 4GB minimum, 8GB recommended
- **Disk Space:** 1GB for application, additional space for database

Software Requirements

- **Operating System:** Ubuntu 20.04 LTS or similar
- **Python:** Version 3.8 or higher
- **PostgreSQL:** Version 12 or higher
- **Redis:** Version 6 or higher
- **Git:** Latest version
- **Postman** or similar API testing tool
- **Selenium** or similar UI testing tool (optional)

Step 1: Create Testing Directory

Create a dedicated directory for testing:

```
mkdir -p ~/saas_testing  
cd ~/saas_testing
```

Step 2: Clone Repository

Clone the repository to the testing directory:

```
git clone https://github.com/sadeqobaid/saas_refactor.git  
cd saas_refactor
```

Step 3: Set Up Python Environment

Create and activate a virtual environment:

```
python3 -m venv test_venv
source test_venv/bin/activate
```

Step 4: Install Dependencies

Install all required packages:

```
pip install fastapi uvicorn sqlalchemy pydantic python-jose[cryptography] passlib python-multipart
pip install pytest pytest-cov httpx
```

Step 5: Set Up Test Database

Create a separate database for testing:

```
# Log in to PostgreSQL
sudo -u postgres psql

# Create test database
CREATE DATABASE saas_platform_test;

# Create test user
CREATE USER test_user WITH ENCRYPTED PASSWORD 'test_password';

# Grant privileges
GRANT ALL PRIVILEGES ON DATABASE saas_platform_test TO test_user;

# Exit PostgreSQL
\q
```

Step 6: Configure Test Environment

Create a `.env.test` file in the root directory:

```
DATABASE_URL=postgresql://test_user:test_password@localhost:5432/saas_platform_test
REDIS_URL=redis://localhost:6379/1
SECRET_KEY=test-secret-key
ALGORITHM=HS256
DB_POOL_SIZE=5
DB_MAX_OVERFLOW=10
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
PASSWORD_RESET_TOKEN_EXPIRE_MINUTES=10
SMTP_SERVER=localhost
SMTP_PORT=1025
SMTP_USERNAME=test
SMTP_PASSWORD=test
EMAIL_FROM=test@example.com
```

Step 7: Set Up Test SMTP Server

For email testing, set up a local SMTP server using Python's built-in smtpd module:

```
python -m smtpd -n -c DebuggingServer localhost:1025
```

This will start a simple SMTP server that prints emails to the console instead of sending them.

Installation Testing

This section covers testing the installation process to ensure all components are correctly installed and configured.

Test Case: Basic Installation Verification

Objective: Verify that the application can be installed from scratch.

Steps:

1. Create a fresh virtual environment:

```
python3 -m venv fresh_test_env  
source fresh_test_env/bin/activate
```

2. Install dependencies:

```
pip install fastapi uvicorn sqlalchemy pydantic python-jose[cryptography] passlib python-dotenv
```

3. Verify installation:

```
pip list | grep -E 'fastapi|uvicorn|sqlalchemy|pydantic|jose|passlib|redis|python-dotenv'
```

Expected Result: All required packages should be listed with their versions.

Verification Method: Check the output of the pip list command to ensure all packages are installed.

Test Case: Database Setup Verification

Objective: Verify that the database can be properly set up.

Steps:

1. Create a test database:

```
sudo -u postgres psql -c "CREATE DATABASE db_setup_test;"  
sudo -u postgres psql -c "CREATE USER setup_test WITH ENCRYPTED PASSWORD 'setup_test';"  
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE db_setup_test TO setup_test;"
```

2. Create a temporary .env file with the test database:

```

echo "DATABASE_URL=postgresql://setup_test:setup_test@localhost:5432/db_setup_test" > .
echo "REDIS_URL=redis://localhost:6379/2" >> .env.setup_test
echo "SECRET_KEY=setup-test-key" >> .env.setup_test
echo "ALGORITHM=HS256" >> .env.setup_test

```

3. Create a simple script to test database connection:

```

# db_test.py
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine, text

# Load environment variables from .env.setup_test
load_dotenv('.env.setup_test')

# Get database URL
database_url = os.getenv('DATABASE_URL')

# Create engine
engine = create_engine(database_url)

# Test connection
with engine.connect() as connection:
    result = connection.execute(text("SELECT 1"))
    print(f"Connection successful: {result.scalar() == 1}")

```

4. Run the test script:

```
python db_test.py
```

Expected Result: The script should output “Connection successful: True”.

Verification Method: Check the output of the script to ensure the database connection is successful.

Test Case: Redis Setup Verification

Objective: Verify that Redis can be properly set up.

Steps:

1. Create a simple script to test Redis connection:

```

# redis_test.py
import os
from dotenv import load_dotenv
import redis

# Load environment variables from .env.setup_test
load_dotenv('.env.setup_test')

```

```

# Get Redis URL
redis_url = os.getenv('REDIS_URL')

# Create Redis client
redis_client = redis.from_url(redis_url)

# Test connection
try:
    redis_client.ping()
    print("Redis connection successful")
except Exception as e:
    print(f"Redis connection failed: {e}")

```

2. Run the test script:

```
python redis_test.py
```

Expected Result: The script should output “Redis connection successful”.

Verification Method: Check the output of the script to ensure the Redis connection is successful.

Test Case: Application Startup Verification

Objective: Verify that the application can start up correctly.

Steps:

1. Create a temporary script to start the application:

```

# startup_test.py
import os
import subprocess
import time
import requests

# Start the application in the background
process = subprocess.Popen(["python", "run.py"], stdout=subprocess.PIPE, stderr=subprocess.PIPE)

# Wait for the application to start
print("Waiting for application to start...")
time.sleep(5)

# Test the health endpoint
try:
    response = requests.get("http://127.0.0.1:8000/health")
    print(f"Health check response: {response.status_code}")
    print(f"Response body: {response.json()}")

```

```

except Exception as e:
    print(f"Health check failed: {e}")

# Terminate the application
process.terminate()
process.wait()

# Print any errors
stdout, stderr = process.communicate()
if stderr:
    print(f"Application errors: {stderr.decode()}")

```

2. Run the test script:

```
python startup_test.py
```

Expected Result: The script should output a successful health check response with status code 200.

Verification Method: Check the output of the script to ensure the application starts up correctly and the health endpoint returns a successful response.

Configuration Testing

This section covers testing the configuration system to ensure that environment variables and settings are correctly loaded and applied.

Test Case: Environment Variable Loading

Objective: Verify that environment variables are correctly loaded from the .env file.

Steps:

1. Create a test .env file:

```
echo "TEST_VAR=test_value" > .env.var_test
```

2. Create a simple script to test environment variable loading:

```

# env_test.py
import os
from dotenv import load_dotenv

# Load environment variables from .env.var_test
load_dotenv('.env.var_test')

# Get test variable
test_var = os.getenv('TEST_VAR')

```

```

# Print result
print(f"TEST_VAR = {test_var}")
print(f"Variable loaded correctly: {test_var == 'test_value'}")

```

3. Run the test script:

```
python env_test.py
```

Expected Result: The script should output “Variable loaded correctly: True”.

Verification Method: Check the output of the script to ensure the environment variable is correctly loaded.

Test Case: Configuration Validation

Objective: Verify that configuration validation works correctly.

Steps:

1. Create a test .env file with invalid values:

```

echo "DATABASE_URL=invalid-url" > .env.invalid
echo "DB_POOL_SIZE=not-a-number" >> .env.invalid

```

2. Create a simple script to test configuration validation:

```

# config_test.py
import os
from dotenv import load_dotenv
from pydantic import BaseSettings, PostgresDsn, validator

# Load environment variables from .env.invalid
load_dotenv('.env.invalid')

# Define a simple settings class
class TestSettings(BaseSettings):
    DATABASE_URL: PostgresDsn
    DB_POOL_SIZE: int

    @validator("DB_POOL_SIZE", pre=True)
    def validate_pool_size(cls, v):
        try:
            return int(v)
        except ValueError:
            raise ValueError("DB_POOL_SIZE must be an integer")

# Try to create settings instance
try:
    settings = TestSettings()
    print("Configuration validation passed (unexpected)")

```



```
except Exception as e:
    print(f"Configuration validation failed as expected: {e}")
```

3. Run the test script:

```
python config_test.py
```

Expected Result: The script should output an error message indicating that the configuration validation failed.

Verification Method: Check the output of the script to ensure the configuration validation correctly identifies invalid values.

Test Case: Default Configuration Values

Objective: Verify that default configuration values are correctly applied when not specified.

Steps:

1. Create a test .env file with minimal configuration:

```
echo "DATABASE_URL=postgresql://test:test@localhost:5432/test" > .env.minimal
```

2. Create a simple script to test default values:

```
# default_test.py
import os
from dotenv import load_dotenv
from pydantic import BaseSettings, PostgresDsn

# Load environment variables from .env.minimal
load_dotenv('.env.minimal')

# Define a simple settings class with defaults
class TestSettings(BaseSettings):
    DATABASE_URL: PostgresDsn
    DB_POOL_SIZE: int = 5
    DB_MAX_OVERFLOW: int = 10
    ALGORITHM: str = "HS256"

# Create settings instance
settings = TestSettings()

# Check default values
print(f"DB_POOL_SIZE = {settings.DB_POOL_SIZE}")
print(f"DB_MAX_OVERFLOW = {settings.DB_MAX_OVERFLOW}")
print(f"ALGORITHM = {settings.ALGORITHM}")
```

```
# Verify defaults
print(f"Default values applied correctly: {settings.DB_POOL_SIZE == 5 and settings.DB_M
```

3. Run the test script:

```
python default_test.py
```

Expected Result: The script should output “Default values applied correctly: True”.

Verification Method: Check the output of the script to ensure the default configuration values are correctly applied.

API Endpoint Testing

This section covers testing the API endpoints to ensure they function correctly and return the expected responses.

Test Case: Health Check Endpoint

Objective: Verify that the health check endpoint returns the correct status.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, test the health check endpoint:

```
curl -X GET http://127.0.0.1:8000/health
```

Expected Result: The endpoint should return a JSON response with status “healthy” and component statuses.

Verification Method: Check the response to ensure it contains the expected fields and values.

Test Case: User Registration

Objective: Verify that users can register successfully.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, create a test tenant:

```
curl -X POST http://127.0.0.1:8000/tenants \
-H "Content-Type: application/json" \
-d '{"name": "Test Tenant", "slug": "test-tenant"}'
```

3. Register a test user:

```
curl -X POST http://127.0.0.1:8000/register \
-H "Content-Type: application/json" \
-H "X-Tenant-ID: test-tenant" \
-d '{"email": "test@example.com", "password": "Test1234!"}'
```

Expected Result: The registration endpoint should return a success message.

Verification Method: Check the response to ensure it indicates successful registration.

Test Case: User Authentication

Objective: Verify that users can authenticate and receive valid tokens.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate with the test user:

```
curl -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=test@example.com&password=Test1234!"
```

Expected Result: The authentication endpoint should return access and refresh tokens.

Verification Method: Check the response to ensure it contains `access_token`, `refresh_token`, and `token_type` fields.

Test Case: Token Refresh

Objective: Verify that refresh tokens can be used to obtain new access tokens.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate with the test user to get tokens:

```
TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=test@example.com&password=Test1234!")
```

```
REFRESH_TOKEN=$(echo $TOKEN_RESPONSE | grep -o '"refresh_token": "[^"]*' | cut -d '"' -f4)
```

3. Use the refresh token to get a new access token:

```
curl -X POST http://127.0.0.1:8000/refresh-token \
-H "Content-Type: application/json" \
-d '{"refresh_token": "$REFRESH_TOKEN"}'
```

Expected Result: The refresh endpoint should return new access and refresh tokens.

Verification Method: Check the response to ensure it contains new access_token and refresh_token fields.

Test Case: Password Reset Request

Objective: Verify that password reset requests are processed correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, request a password reset:

```
curl -X POST http://127.0.0.1:8000/reset-password/request \
-H "Content-Type: application/json" \
-H "X-Tenant-ID: test-tenant" \
-d '{"email": "test@example.com"}'
```

Expected Result: The endpoint should return a success message.

Verification Method: Check the response to ensure it indicates that a reset link will be sent if the email is registered.

Test Case: Tenant Creation (Super Admin Only)

Objective: Verify that super admins can create new tenants.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a super admin:

```
ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: system" \
-d "username=admin@example.com&password=Admin1234!")
```

```
ADMIN_ACCESS_TOKEN=$(echo $ADMIN_TOKEN_RESPONSE | grep -o '"access_token": "[^"]*' | cut -d '"' -f 2)
```

3. Create a new tenant:

```
curl -X POST http://127.0.0.1:8000/tenants \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $ADMIN_ACCESS_TOKEN" \
-d '{"name": "New Tenant", "slug": "new-tenant", "status": "active"}'
```

Expected Result: The endpoint should return the created tenant details.

Verification Method: Check the response to ensure it contains the tenant information with the correct name, slug, and status.

Test Case: Tenant Configuration

Objective: Verify that tenant configurations can be created and retrieved.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a tenant admin:

```
TENANT_ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=admin@test-tenant.com&password=Admin1234!")
```

```
TENANT_ADMIN_ACCESS_TOKEN=$(echo $TENANT_ADMIN_TOKEN_RESPONSE | grep -o '"access_token"')
```

3. Create a tenant configuration:

```
curl -X POST http://127.0.0.1:8000/tenants/test-tenant/config \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TENANT_ADMIN_ACCESS_TOKEN" \
-d '{"key": "max_users", "value": "100"}'
```

4. Retrieve the tenant configuration:

```
curl -X GET http://127.0.0.1:8000/tenants/test-tenant/config/max_users \
-H "Authorization: Bearer $TENANT_ADMIN_ACCESS_TOKEN"
```

Expected Result: The endpoint should return the configuration value.

Verification Method: Check the response to ensure it contains the correct configuration value.

Test Case: Statistics Endpoints

Objective: Verify that statistics endpoints return the correct data.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a tenant admin:

```
TENANT_ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=admin@test-tenant.com&password=Admin1234!")
```

```
TENANT_ADMIN_ACCESS_TOKEN=$(echo $TENANT_ADMIN_TOKEN_RESPONSE | grep -o '"access_token"')
```

3. Get Monthly Active Users statistics:

```
curl -X GET http://127.0.0.1:8000/admin/stats/mau \
-H "Authorization: Bearer $TENANT_ADMIN_ACCESS_TOKEN" \
-H "X-Tenant-ID: test-tenant"
```

4. Get usage statistics:

```
curl -X GET http://127.0.0.1:8000/admin/stats/usage \
-H "Authorization: Bearer $TENANT_ADMIN_ACCESS_TOKEN" \
-H "X-Tenant-ID: test-tenant"
```

Expected Result: The endpoints should return statistics data in the expected format.

Verification Method: Check the responses to ensure they contain the expected fields and data types.

Frontend Testing

This section covers testing the Streamlit frontend to ensure it functions correctly and provides a good user experience.

Test Case: Streamlit Application Startup

Objective: Verify that the Streamlit application starts correctly.

Steps:

1. Start the backend API:

```
python run.py
```

2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```

3. Open a web browser and navigate to <http://localhost:8501>

Expected Result: The Streamlit application should load and display the login/register interface.

Verification Method: Visually verify that the application loads correctly and displays the expected interface.

Test Case: User Registration via UI

Objective: Verify that users can register through the Streamlit UI.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```
3. Open a web browser and navigate to <http://localhost:8501>
4. Select “Register” from the sidebar
5. Enter a test email and password:
 - Email: ui-test@example.com
 - Password: UiTest1234!
6. Click the “Register” button

Expected Result: The UI should display a success message indicating that registration was successful.

Verification Method: Visually verify that the success message is displayed and that no errors occur.

Test Case: User Login via UI

Objective: Verify that users can log in through the Streamlit UI.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```
3. Open a web browser and navigate to <http://localhost:8501>
4. Select “Login” from the sidebar
5. Enter the test credentials:
 - Email: ui-test@example.com
 - Password: UiTest1234!

6. Click the “Login” button

Expected Result: The UI should display a success message and redirect to the Dashboard.

Verification Method: Visually verify that login is successful and the Dashboard is displayed.

Test Case: Password Reset via UI

Objective: Verify that users can request password resets through the Streamlit UI.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```
3. Open a web browser and navigate to <http://localhost:8501>
4. Select “Reset Password” from the sidebar
5. In the “Request Reset” tab, enter the test email:
 - Email: `ui-test@example.com`
6. Click the “Send Reset Link” button

Expected Result: The UI should display a message indicating that a reset link will be sent if the email is registered.

Verification Method: Visually verify that the message is displayed and that no errors occur.

Test Case: Dashboard Display

Objective: Verify that the Dashboard displays correctly after login.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```
3. Open a web browser and navigate to <http://localhost:8501>
4. Log in with valid credentials

5. Observe the Dashboard

Expected Result: The Dashboard should display user information, tenant information, and other relevant data.

Verification Method: Visually verify that the Dashboard displays the expected information.

Test Case: Password Change via UI

Objective: Verify that users can change their passwords through the Streamlit UI.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```
3. Open a web browser and navigate to <http://localhost:8501>
4. Log in with valid credentials
5. Select “Change Password” from the sidebar
6. Enter the current password and a new password:
 - Current Password: UiTest1234!
 - New Password: UiTest5678!
 - Confirm Password: UiTest5678!
7. Click the “Change Password” button

Expected Result: The UI should display a success message indicating that the password was changed.

Verification Method: Visually verify that the success message is displayed and that no errors occur.

Test Case: Logout via UI

Objective: Verify that users can log out through the Streamlit UI.

Steps:

1. Start the backend API:

```
python run.py
```
2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```

3. Open a web browser and navigate to `http://localhost:8501`
4. Log in with valid credentials
5. Select “Logout” from the sidebar

Expected Result: The UI should log the user out and return to the login/register interface.

Verification Method: Visually verify that logout is successful and the login/register interface is displayed.

Multi-tenancy Testing

This section covers testing the multi-tenancy features to ensure proper data isolation and tenant-specific functionality.

Test Case: Tenant Isolation

Objective: Verify that data is properly isolated between tenants.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, create two test tenants:

```
# Create Tenant A
curl -X POST http://127.0.0.1:8000/tenants \
  -H "Content-Type: application/json" \
  -d '{"name": "Tenant A", "slug": "tenant-a"}'
```

```
# Create Tenant B
curl -X POST http://127.0.0.1:8000/tenants \
  -H "Content-Type: application/json" \
  -d '{"name": "Tenant B", "slug": "tenant-b"}'
```

3. Register a user in each tenant:

```
# Register user in Tenant A
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: tenant-a" \
  -d '{"email": "user@example.com", "password": "Test1234!"}'
```

```
# Register user in Tenant B
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: tenant-b" \
  -d '{"email": "user@example.com", "password": "Test1234!"}'
```

4. Authenticate as the user in Tenant A:

```
curl -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: tenant-a" \
-d "username=user@example.com&password=Test1234!"
```

5. Authenticate as the user in Tenant B:

```
curl -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: tenant-b" \
-d "username=user@example.com&password=Test1234!"
```

Expected Result: Both authentication requests should succeed, demonstrating that the same email can be used in different tenants.

Verification Method: Check the responses to ensure both authentication requests return valid tokens.

Test Case: Tenant Identification

Objective: Verify that tenant identification works correctly with both header and query parameter.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, test tenant identification with header:

```
curl -X GET http://127.0.0.1:8000/health \
-H "X-Tenant-ID: tenant-a"
```

3. Test tenant identification with query parameter:

```
curl -X GET "http://127.0.0.1:8000/health?tenant=tenant-a"
```

Expected Result: Both requests should succeed and identify the tenant correctly.

Verification Method: Check the responses to ensure both requests return successful responses.

Test Case: Tenant-specific Configuration

Objective: Verify that tenant-specific configurations work correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a tenant admin for Tenant A:

```
TENANT_A_ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: tenant-a" \
-d "username=admin@tenant-a.com&password=Admin1234!")

TENANT_A_ADMIN_ACCESS_TOKEN=$(echo $TENANT_A_ADMIN_TOKEN_RESPONSE | grep -o '"access_token": ' | cut -d'"' -f2)
```

3. Create a configuration for Tenant A:

```
curl -X POST http://127.0.0.1:8000/tenants/tenant-a/config \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TENANT_A_ADMIN_ACCESS_TOKEN" \
-d '{"key": "theme", "value": "dark"}'
```

4. Authenticate as a tenant admin for Tenant B:

```
TENANT_B_ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: tenant-b" \
-d "username=admin@tenant-b.com&password=Admin1234!")

TENANT_B_ADMIN_ACCESS_TOKEN=$(echo $TENANT_B_ADMIN_TOKEN_RESPONSE | grep -o '"access_token": ' | cut -d'"' -f2)
```

5. Create a configuration for Tenant B:

```
curl -X POST http://127.0.0.1:8000/tenants/tenant-b/config \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TENANT_B_ADMIN_ACCESS_TOKEN" \
-d '{"key": "theme", "value": "light"}'
```

6. Retrieve the configuration for Tenant A:

```
curl -X GET http://127.0.0.1:8000/tenants/tenant-a/config/theme \
-H "Authorization: Bearer $TENANT_A_ADMIN_ACCESS_TOKEN"
```

7. Retrieve the configuration for Tenant B:

```
curl -X GET http://127.0.0.1:8000/tenants/tenant-b/config/theme \
-H "Authorization: Bearer $TENANT_B_ADMIN_ACCESS_TOKEN"
```

Expected Result: The configuration values should be different for each tenant.

Verification Method: Check the responses to ensure Tenant A returns “dark” and Tenant B returns “light”.

Test Case: Tenant Status Management

Objective: Verify that tenant status management works correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a super admin:

```
SUPER_ADMIN_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: system" \
-d "username=admin@example.com&password=Admin1234!")
```

```
SUPER_ADMIN_ACCESS_TOKEN=$(echo $SUPER_ADMIN_TOKEN_RESPONSE | grep -o "access_token":")
```

3. Update Tenant A status to “suspended”:

```
curl -X PUT http://127.0.0.1:8000/tenants/tenant-a \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $SUPER_ADMIN_ACCESS_TOKEN" \
-d '{"status": "suspended"}'
```

4. Try to authenticate as a user in Tenant A:

```
curl -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: tenant-a" \
-d "username=user@example.com&password=Test1234!"
```

Expected Result: The authentication request should fail because the tenant is suspended.

Verification Method: Check the response to ensure it returns an error indicating that the tenant is suspended.

Database Testing

This section covers testing database operations to ensure data integrity and proper functioning of database-related features.

Test Case: Database Schema Verification

Objective: Verify that the database schema is correctly created.

Steps:

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. List all tables:

```
\dt
```

3. Describe each table:

```
\d tenants
\d tenant_configs
\d users
\d refresh_tokens
\d password_reset_tokens
\d user_activities
\d monthly_active_users
\d usage_summaries
```

Expected Result: All tables should exist with the correct columns, constraints, and indexes.

Verification Method: Check the output to ensure all tables have the expected structure.

Test Case: Foreign Key Constraints

Objective: Verify that foreign key constraints work correctly.

Steps:

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. Try to insert a user with a non-existent tenant_id:

```
INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('test@example.com', 'hash', 'basic_user', 999);
```

Expected Result: The insert should fail with a foreign key constraint violation.

Verification Method: Check the error message to ensure it indicates a foreign key constraint violation.

Test Case: Unique Constraints

Objective: Verify that unique constraints work correctly.

Steps:

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. Insert a tenant:

```
INSERT INTO tenants (name, slug, status)
VALUES ('Test Tenant', 'test-tenant', 'active');
```

3. Try to insert another tenant with the same slug:

```
INSERT INTO tenants (name, slug, status)
VALUES ('Another Tenant', 'test-tenant', 'active');
```

Expected Result: The second insert should fail with a unique constraint violation.

Verification Method: Check the error message to ensure it indicates a unique constraint violation.

Test Case: Composite Unique Constraints

Objective: Verify that composite unique constraints work correctly.

Steps:

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. Insert a tenant:

```
INSERT INTO tenants (name, slug, status)
VALUES ('Test Tenant', 'test-tenant', 'active');
```

3. Insert a user:

```
INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('test@example.com', 'hash', 'basic_user', 1);
```

4. Try to insert another user with the same email in the same tenant:

```
INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('test@example.com', 'hash', 'basic_user', 1);
```

5. Insert a user with the same email in a different tenant:

```
INSERT INTO tenants (name, slug, status)
VALUES ('Another Tenant', 'another-tenant', 'active');

INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('test@example.com', 'hash', 'basic_user', 2);
```

Expected Result: The second insert should fail with a unique constraint violation, but the third insert should succeed.

Verification Method: Check the error message for the second insert and the success of the third insert.

Test Case: Cascade Delete

Objective: Verify that cascade delete works correctly.

Steps:

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. Insert a tenant and related records:

```
INSERT INTO tenants (name, slug, status)
VALUES ('Test Tenant', 'test-tenant', 'active');

INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('test@example.com', 'hash', 'basic_user', 1);

INSERT INTO tenant_configs (tenant_id, key, value)
VALUES (1, 'theme', 'dark');
```

3. Delete the tenant:

```
DELETE FROM tenants WHERE id = 1;
```

4. Check if related records were deleted:

```
SELECT * FROM users WHERE tenant_id = 1;
SELECT * FROM tenant_configs WHERE tenant_id = 1;
```

Expected Result: The related records should be deleted when the tenant is deleted.

Verification Method: Check that the queries return no results after the tenant is deleted.

Security Testing

This section covers testing security features to ensure the system is protected against common vulnerabilities.

Test Case: Password Hashing

Objective: Verify that passwords are properly hashed and not stored in plain text.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, register a test user:

```
curl -X POST http://127.0.0.1:8000/register \
-H "Content-Type: application/json" \
-H "X-Tenant-ID: test-tenant" \
-d '{"email": "security-test@example.com", "password": "Security1234!"}'
```

3. Connect to the database and check the password hash:


```
sudo -u postgres psql -d saas_platform_test -c "SELECT email, password_hash FROM users"
```

Expected Result: The password should be stored as a hash, not in plain text.

Verification Method: Check that the password_hash column contains a hashed value, not the original password.

Test Case: Password Complexity Requirements

Objective: Verify that password complexity requirements are enforced.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, try to register a user with a weak password:

```
# Too short
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: test-tenant" \
  -d '{"email": "weak-password@example.com", "password": "Weak1!"}'

# No uppercase
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: test-tenant" \
  -d '{"email": "weak-password@example.com", "password": "weak1234!"}'

# No number
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: test-tenant" \
  -d '{"email": "weak-password@example.com", "password": "WeakPassword!"}'

# No special character
curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: test-tenant" \
  -d '{"email": "weak-password@example.com", "password": "WeakPassword1"}'
```

Expected Result: All registration attempts should fail with appropriate error messages.

Verification Method: Check the responses to ensure they contain error messages indicating the password complexity requirements.

Test Case: JWT Token Validation

Objective: Verify that JWT tokens are properly validated.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate to get a valid token:

```
TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=security-test@example.com&password=Security1234!")

ACCESS_TOKEN=$(echo $TOKEN_RESPONSE | grep -o '"access_token": "[^"]*' | cut -d'"' -f4)
```

3. Try to access a protected endpoint with an invalid token:

```
curl -X GET http://127.0.0.1:8000/admin/stats/mau \
-H "Authorization: Bearer invalid.token.here" \
-H "X-Tenant-ID: test-tenant"
```

4. Try to access a protected endpoint with a valid token but wrong tenant:

```
curl -X GET http://127.0.0.1:8000/admin/stats/mau \
-H "Authorization: Bearer $ACCESS_TOKEN" \
-H "X-Tenant-ID: wrong-tenant"
```

Expected Result: Both requests should fail with authentication errors.

Verification Method: Check the responses to ensure they contain appropriate error messages.

Test Case: Role-Based Access Control

Objective: Verify that role-based access control works correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, authenticate as a basic user:

```
BASIC_USER_TOKEN_RESPONSE=$(curl -s -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: test-tenant" \
-d "username=security-test@example.com&password=Security1234!")

BASIC_USER_ACCESS_TOKEN=$(echo $BASIC_USER_TOKEN_RESPONSE | grep -o '"access_token": "[^"]'
```

3. Try to access an admin-only endpoint:

```
curl -X GET http://127.0.0.1:8000/admin/stats/mau \
-H "Authorization: Bearer $BASIC_USER_ACCESS_TOKEN" \
-H "X-Tenant-ID: test-tenant"
```

4. Try to access a super-admin-only endpoint:

```
curl -X GET http://127.0.0.1:8000/super-admin/stats/tenants \
-H "Authorization: Bearer $BASIC_USER_ACCESS_TOKEN" \
-H "X-Tenant-ID: test-tenant"
```

Expected Result: Both requests should fail with permission errors.

Verification Method: Check the responses to ensure they contain appropriate error messages indicating insufficient permissions.

Test Case: Rate Limiting

Objective: Verify that rate limiting works correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, make multiple rapid requests to a rate-limited endpoint:

```
for i in {1..10}; do
  curl -X POST http://127.0.0.1:8000/register \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: test-tenant" \
  -d '{"email": "rate-limit-test'$i'@example.com", "password": "RateLimit1234!"}'
done
```

Expected Result: After a certain number of requests, the rate limit should be exceeded.

Verification Method: Check the responses to ensure that later requests return rate limit exceeded errors.

Performance Testing

This section covers testing the performance of the system under various load conditions.

Test Case: Response Time Measurement

Objective: Measure the response time of key endpoints.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, create a script to measure response times:

```
# response_time_test.py
import time
import requests
import statistics

# Endpoints to test
endpoints = [
    "http://127.0.0.1:8000/health",
    "http://127.0.0.1:8000/"
]

# Number of requests per endpoint
num_requests = 10

results = {}

for endpoint in endpoints:
    times = []
    for i in range(num_requests):
        start_time = time.time()
        response = requests.get(endpoint)
        end_time = time.time()
        response_time = (end_time - start_time) * 1000 # Convert to milliseconds
        times.append(response_time)

    avg_time = statistics.mean(times)
    min_time = min(times)
    max_time = max(times)
    median_time = statistics.median(times)

    results[endpoint] = {
        "average": avg_time,
        "min": min_time,
        "max": max_time,
        "median": median_time
    }

# Print results
for endpoint, metrics in results.items():
    print(f"Endpoint: {endpoint}")
    print(f"  Average response time: {metrics['average']:.2f} ms")
```

```

print(f"  Min response time: {metrics['min']:.2f} ms")
print(f"  Max response time: {metrics['max']:.2f} ms")
print(f"  Median response time: {metrics['median']:.2f} ms")
print()

```

3. Run the script:

```
python response_time_test.py
```

Expected Result: The script should output response time metrics for each endpoint.

Verification Method: Check the output to ensure response times are within acceptable limits (e.g., < 200ms for simple endpoints).

Test Case: Connection Pool Testing

Objective: Verify that the database connection pool handles multiple concurrent connections correctly.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, create a script to simulate multiple concurrent database operations:

```

# connection_pool_test.py
import concurrent.futures
import requests
import time

# Number of concurrent requests
num_concurrent = 20

# Total number of requests
num_requests = 100

# Endpoint that requires database access
endpoint = "http://127.0.0.1:8000/health"

def make_request(i):
    try:
        start_time = time.time()
        response = requests.get(endpoint)
        end_time = time.time()
        response_time = (end_time - start_time) * 1000 # Convert to milliseconds
        return {

```

```

        "request_id": i,
        "status_code": response.status_code,
        "response_time": response_time
    }
except Exception as e:
    return {
        "request_id": i,
        "error": str(e)
    }

# Use ThreadPoolExecutor to make concurrent requests
with concurrent.futures.ThreadPoolExecutor(max_workers=num_concurrent) as executor:
    futures = [executor.submit(make_request, i) for i in range(num_requests)]
    results = [future.result() for future in concurrent.futures.as_completed(futures)]

# Analyze results
successful_requests = [r for r in results if "status_code" in r and r["status_code"] == 200]
failed_requests = [r for r in results if "status_code" not in r or r["status_code"] != 200]

# Print summary
print(f"Total requests: {num_requests}")
print(f"Successful requests: {len(successful_requests)}")
print(f"Failed requests: {len(failed_requests)}")

if successful_requests:
    response_times = [r["response_time"] for r in successful_requests]
    avg_time = sum(response_times) / len(response_times)
    min_time = min(response_times)
    max_time = max(response_times)
    print(f"Average response time: {avg_time:.2f} ms")
    print(f"Min response time: {min_time:.2f} ms")
    print(f"Max response time: {max_time:.2f} ms")

if failed_requests:
    print("\nFailed requests:")
    for r in failed_requests:
        print(f"Request {r['request_id']}: {r.get('error', f'Status code: {r.get('status_code', 'N/A')}')")

```

3. Run the script:

```
python connection_pool_test.py
```

Expected Result: Most or all requests should succeed, demonstrating that the connection pool can handle multiple concurrent connections.

Verification Method: Check the output to ensure a high success rate and reasonable response times.

Test Case: Memory Usage Monitoring

Objective: Monitor memory usage during operation to identify potential memory leaks.

Steps:

1. Start the application with memory profiling:

```
python -m memory_profiler run.py
```

2. In a separate terminal, create a script to generate load:

```
# generate_load.py
import requests
import time

# Number of requests
num_requests = 1000

# Endpoint to test
endpoint = "http://127.0.0.1:8000/health"

for i in range(num_requests):
    try:
        response = requests.get(endpoint)
        print(f"Request {i+1}/{num_requests}: Status {response.status_code}")
    except Exception as e:
        print(f"Request {i+1}/{num_requests}: Error - {str(e)}")

    # Small delay to prevent overwhelming the server
    time.sleep(0.01)
```

3. Run the script:

```
python generate_load.py
```

4. Monitor memory usage in the application terminal.

Expected Result: Memory usage should remain stable or increase only slightly over time.

Verification Method: Check the memory profiling output to ensure there are no significant memory leaks.

Integration Testing

This section covers testing the integration between different components of the system.

Test Case: API and Database Integration

Objective: Verify that the API correctly interacts with the database.

Steps:

1. Start the application:

```
python run.py
```

2. In a separate terminal, create a test tenant:

```
curl -X POST http://127.0.0.1:8000/tenants \
-H "Content-Type: application/json" \
-d '{"name": "Integration Test Tenant", "slug": "integration-test"}
```

3. Register a test user:

```
curl -X POST http://127.0.0.1:8000/register \
-H "Content-Type: application/json" \
-H "X-Tenant-ID: integration-test" \
-d '{"email": "integration-test@example.com", "password": "Integration1234!"}'
```

4. Connect to the database and verify the data was inserted:

```
sudo -u postgres psql -d saas_platform_test -c "SELECT * FROM tenants WHERE slug = 'integration-test'"
sudo -u postgres psql -d saas_platform_test -c "SELECT * FROM users WHERE email = 'integration-test@example.com'"
```

Expected Result: The database queries should return the tenant and user that were created through the API.

Verification Method: Check the query results to ensure the data matches what was sent through the API.

Test Case: API and Frontend Integration

Objective: Verify that the Streamlit frontend correctly interacts with the API.

Steps:

1. Start the backend API:

```
python run.py
```

2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```

3. Open a web browser and navigate to <http://localhost:8501>

4. Register a new user through the UI:

- Email: frontend-integration@example.com
- Password: Frontend1234!

5. In a terminal, check if the user was created in the database:


```
sudo -u postgres psql -d saas_platform_test -c "SELECT * FROM users WHERE email = 'from"
```

Expected Result: The database query should return the user that was created through the frontend.

Verification Method: Check the query result to ensure the data matches what was entered in the frontend.

Test Case: Email Integration

Objective: Verify that the email sending functionality works correctly.

Steps:

1. Start the test SMTP server:

```
python -m smtpd -n -c DebuggingServer localhost:1025
```

2. In a separate terminal, start the application:

```
python run.py
```

3. In another terminal, request a password reset:

```
curl -X POST http://127.0.0.1:8000/reset-password/request \
-H "Content-Type: application/json" \
-H "X-Tenant-ID: integration-test" \
-d '{"email": "integration-test@example.com"}'
```

4. Check the SMTP server output for the email.

Expected Result: The SMTP server should receive and display the password reset email.

Verification Method: Check the SMTP server output to ensure the email was sent with the correct content.

Regression Testing

This section covers testing to ensure that new changes don't break existing functionality.

Test Case: API Backward Compatibility

Objective: Verify that API changes maintain backward compatibility.

Steps:

1. Document the current API responses:

```
# Create a directory for API response snapshots
mkdir -p api_snapshots

# Get and save responses for key endpoints
```

```
curl -s -X GET http://127.0.0.1:8000/health > api_snapshots/health.json
curl -s -X GET http://127.0.0.1:8000/ > api_snapshots/root.json
```

2. After making changes to the codebase, start the updated application:

```
python run.py
```

3. Compare the new responses with the snapshots:

```
# Get new responses
curl -s -X GET http://127.0.0.1:8000/health > api_snapshots/health_new.json
curl -s -X GET http://127.0.0.1:8000/ > api_snapshots/root_new.json

# Compare responses
diff api_snapshots/health.json api_snapshots/health_new.json
diff api_snapshots/root.json api_snapshots/root_new.json
```

Expected Result: The responses should be identical or have only expected differences.

Verification Method: Check the diff output to ensure there are no unexpected changes in the API responses.

Test Case: Database Migration Testing

Objective: Verify that database migrations work correctly and preserve data.

Steps:

1. Create a backup of the test database:

```
sudo -u postgres pg_dump saas_platform_test > db_backup.sql
```

2. Apply database migrations:

```
# Example migration script
python migrate.py
```

3. Verify that the data is preserved:

```
sudo -u postgres psql -d saas_platform_test -c "SELECT COUNT(*) FROM users;"
sudo -u postgres psql -d saas_platform_test -c "SELECT COUNT(*) FROM tenants;"
```

4. Test key functionality to ensure it still works after migration:

```
# Test authentication
curl -X POST http://127.0.0.1:8000/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "X-Tenant-ID: integration-test" \
-d "username=integration-test@example.com&password=Integration1234!"
```

Expected Result: The data should be preserved and functionality should continue to work after migration.

Verification Method: Check the query results and API responses to ensure data integrity and functionality.

Test Case Templates

This section provides templates for creating new test cases.

API Endpoint Test Template

Test Case: [Endpoint Name]

****Objective**:** Verify that the [Endpoint Name] endpoint functions correctly.

****Steps**:**

1. Start the application:

```
```bash
python run.py
```

2. In a separate terminal, describe the test steps:

```
curl -X [METHOD] http://127.0.0.1:8000/[endpoint-path] \
-H "Content-Type: application/json" \
-H "Authorization: Bearer [token]" \
-d '[request-body]'
```

**Expected Result:** [Describe the expected result]

**Verification Method:** [Describe how to verify the result]

### Frontend Feature Test Template

**Test Case:** [Feature Name]

**Objective:** Verify that the [Feature Name] feature functions correctly in the UI.

**Steps:**

1. Start the backend API:

```
python run.py
```

2. In a separate terminal, start the Streamlit application:

```
streamlit run streamlit_app.py
```

3. Open a web browser and navigate to <http://localhost:8501>

4. [Describe the test steps in the UI]

**Expected Result:** [Describe the expected result]

**Verification Method:** [Describe how to verify the result]

### Database Test Template

**Test Case:** [Database Feature]

**Objective:** Verify that [Database Feature] works correctly.

**Steps:**

1. Connect to the test database:

```
sudo -u postgres psql -d saas_platform_test
```

2. [SQL queries] ““

**Expected Result:** [Describe the expected result]

**Verification Method:** [Describe how to verify the result]

## Automated Testing

This section covers setting up and running automated tests for the system.

### Setting Up Pytest

1. Create a `tests` directory:

```
```bash
mkdir -p tests
touch tests/__init__.py
```

2. Create a `conftest.py` file for test fixtures:

```
# tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.db.database import Base
from app.main import app
from app.dependencies import get_db
```

```
# Test database URL
```

```
TEST_DATABASE_URL = "postgresql://test_user:test_password@localhost:5432/saas_platform_
```

```
# Create test engine
```

```
engine = create_engine(TEST_DATABASE_URL)
```

```
# Create test session
```

```
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```

@pytest.fixture
def test_db():
    # Create tables
    Base.metadata.create_all(bind=engine)

    # Create session
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

    # Drop tables
    Base.metadata.drop_all(bind=engine)

@pytest.fixture
def client(test_db):
    # Override the get_db dependency
    def override_get_db():
        try:
            yield test_db
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db

    # Create test client
    with TestClient(app) as client:
        yield client

    # Remove override
    app.dependency_overrides = {}

```

3. Create a test file for the health endpoint:

```

# tests/test_health.py
def test_health_endpoint(client):
    response = client.get("/health")
    assert response.status_code == 200
    data = response.json()
    assert "status" in data
    assert "components" in data
    assert "database" in data["components"]

```

4. Create a test file for user authentication:

```

# tests/test_auth.py
from app.models import Tenant, User

```

```

def test_user_registration(client, test_db):
    # Create a test tenant
    tenant = Tenant(name="Test Tenant", slug="test-tenant", status="active")
    test_db.add(tenant)
    test_db.commit()

    # Register a user
    response = client.post(
        "/register",
        headers={"X-Tenant-ID": "test-tenant"},
        json={"email": "test@example.com", "password": "Test1234!"}
    )

    assert response.status_code == 200
    assert response.json() == {"message": "User registered successfully"}

    # Verify user was created
    user = test_db.query(User).filter(User.email == "test@example.com").first()
    assert user is not None
    assert user.email == "test@example.com"
    assert user.tenant_id == tenant.id

def test_user_authentication(client, test_db):
    # Create a test tenant
    tenant = Tenant(name="Test Tenant", slug="test-tenant", status="active")
    test_db.add(tenant)
    test_db.commit()

    # Register a user
    client.post(
        "/register",
        headers={"X-Tenant-ID": "test-tenant"},
        json={"email": "auth-test@example.com", "password": "Auth1234!"}
    )

    # Authenticate
    response = client.post(
        "/token",
        headers={"X-Tenant-ID": "test-tenant"},
        data={"username": "auth-test@example.com", "password": "Auth1234!"}
    )

    assert response.status_code == 200
    data = response.json()
    assert "access_token" in data

```

```

assert "refresh_token" in data
assert "token_type" in data
assert data["token_type"] == "bearer"

```

5. Run the tests:

```
pytest tests/
```

Setting Up Coverage Reporting

1. Install pytest-cov:

```
pip install pytest-cov
```

2. Run tests with coverage:

```
pytest --cov=app tests/
```

3. Generate a coverage report:

```
pytest --cov=app --cov-report=html tests/
```

4. Open the coverage report:

```
open htmlcov/index.html
```

Setting Up Continuous Integration

1. Create a `.github/workflows/test.yml` file:

```

name: Test

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:12
        env:
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
          POSTGRES_DB: saas_platform_test
        ports:
          - 5432:5432

```

```

    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5

redis:
  image: redis:6
  ports:
    - 6379:6379
  options: >-
    --health-cmd "redis-cli ping"
    --health-interval 10s
    --health-timeout 5s
    --health-retries 5

steps:
- uses: actions/checkout@v2

- name: Set up Python
  uses: actions/setup-python@v2
  with:
    python-version: '3.8'

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
    pip install pytest pytest-cov

- name: Run tests
  run: |
    pytest --cov=app tests/
  env:
    DATABASE_URL: postgresql://test_user:test_password@localhost:5432/saas_platform
    REDIS_URL: redis://localhost:6379/0
    SECRET_KEY: test-secret-key
    ALGORITHM: HS256

```

2. Commit and push the workflow file to enable CI testing on GitHub.

Load Testing with Locust

1. Install Locust:

```
pip install locust
```


2. Create a `locustfile.py`:

```
# locustfile.py
from locust import HttpUser, task, between

class SaaSPlatformUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        # Create a tenant if it doesn't exist
        self.client.post("/tenants", json={"name": "Load Test Tenant", "slug": "load-te

        # Register a user
        self.client.post(
            "/register",
            headers={"X-Tenant-ID": "load-test"},
            json={"email": f"user_{self.user_id}@example.com", "password": "LoadTest123
        )

        # Authenticate
        response = self.client.post(
            "/token",
            headers={"X-Tenant-ID": "load-test"},
            data={"username": f"user_{self.user_id}@example.com", "password": "LoadTest
        )

        data = response.json()
        self.token = data["access_token"]

    @task
    def health_check(self):
        self.client.get("/health")

    @task
    def root_endpoint(self):
        self.client.get("/")

    @task
    def get_stats(self):
        self.client.get(
            "/admin/stats/mau",
            headers={
                "Authorization": f"Bearer {self.token}",
                "X-Tenant-ID": "load-test"
            }
        )
```

3. Run Locust:

```
locust -H http://127.0.0.1:8000
```

4. Open the Locust web interface at <http://localhost:8089> and start a load test.